Parallel Three-sequence Alignment with Space-efficient

Chen Tai Huang[1], Chun Yuan Lin[2], Yeh-Ching Chung[1], and Chuan Yi Tang[1,*]

[1]Department of Computer Science and [2]Institute of Molecular and Cellular Biology
National Tsing Hua University, Hsin-chu, Taiwan 300, ROC
{g936325@oz, cyulin@mx, ychung@cs, *cytang@cs}.nthu.edu.tw

Abstract

The sequence alignment is a fundamental problem in computational biology. Many sequence alignment methods have been proposed, such as pair-wise sequence alignment, multiple sequence alignment (MSA), syntenic alignment, constraint multiple sequence alignment, and etc. Among them, pair-wise sequence alignment is a most commonly used method. Recently, MSA is more and more important which has been used to solve many molecular biological problems. The sum-of-pairs MSA problem has been proved to be a NP-complete problem. A progressive algorithm was used to solve MSA problem which is based on the pair-wise sequence alignments. However, this progressive MSA has a problem. This problem may be solved by using three-sequence alignment as a basic step instead of pair-wise sequence alignment. Three-sequence alignment can be solved sequentially in O(mnl) time complexity and O(mn) space complexity, where m, n and l are the lengths of the sequences to be aligned. The complexities of three-sequence alignment limit its applicability. Hence, to reduce the complexities becomes an important issue. In this paper, we propose a parallel algorithm to solve this problem. Our parallel algorithm requires O((mn)/p) space complexity and O((mnl)/p) time complexity, where p is the number of processors. Both experimental results and theoretical analysis are presented. The experimental results show that our parallel algorithm is applicable and achieves a good speed up.

1 Introduction

The sequence alignment is a fundamental problem in computational biology. High sequence similarity usually implies functional or structural similarity, researchers use sequence comparison and alignment tools to relate the molecular structure and function to sequences. Many sequence alignment methods have been proposed such as pair-wise sequence alignment, multiple sequence alignment (*MSA*), syntenic alignment, constraint multiple sequence alignment, and etc. Among them, pair-wise sequence alignment is a most commonly used method.

Sequence alignment algorithms typically use the dynamic programming in which one or multiple tables are filled through a scoring mechanism. Once the best score in the tables is found a trace back procedure is involved for finding the optimal alignment.

Let m and n be the lengths of two sequences to be aligned. Several tables with of size $(m+1)\times(n+1)$ are filled for finding an optimal alignment of pair-wise sequence alignment. It takes O(mn) time complexity and requires O(mn) space complexity. Myers and Miller [11] applied the divide-and-conquer technique of Hirschberg [7] to reduce the space requirement to linear space. Huang [10] extended the Myers and Miller's algorithm [11] to three-sequence alignment one. Let m, n and l be the lengths of three sequences to be aligned. The three-sequence alignment algorithm is done by filling several tables with of size $(m+1)\times(n+1)\times(l+1)$. It takes O($mnl$) time complexity and requires O(mn) space complexity.

MSA is an useful tool in the phylogenetic analysis among various organisms, the identification of conserved motifs and domains in a group of related proteins, the secondary and tertiary structure prediction of a protein or RNA [3-5, 12, 13], and etc. The sum-of-pairs *MSA* problem has been proved to be a NP-complete problem [2, 16]. A progressive algorithm is used to solve *MSA* problem which is based on pair-wise sequence alignments. However, this progressive *MSA* has a problem: It may be not consistent for certain positions. This problem may be solved by using three-sequence alignment as a basic step instead of pair-wise sequence alignment. However, although the space requirement is reduced to quadratic space, the time and space complexities of three-sequence alignment limit its applicability. Hence, to reduce the time and space complexities of three-sequence alignment becomes an important issue.

Use of parallel computers is a good way to reduce the time and space requirements. In this paper, we propose a parallel algorithm to reduce the computational cost. Our parallel algorithm requires O((mn)/p) space complexity and O((mnl)/p) time complexity which are optimal, where p is the number of processors. Besides, load balancing is also taken into account. Our algorithm has been implemented as a *MPI+C* program. Both experimental results and theoretical analysis are presented. In the experiments,

*The corresponding author.

a random data set is used for testing our parallel algorithm. The experimental results show that a good performance and a good speed-up are achieved by our parallel algorithm, which is more applicable than the sequential version.

The rest of the paper is organized as follows: In Section 2, a brief survey of earlier efforts is presented. Section 3 describes the problem of three-sequence alignment and our parallel approach. In Section 4, theoretical analysis of our method is given. In Section 5, the experimental results are presented.

2 Related work

Sequence alignment is an important problem in molecular biology. Many methods for reducing the time and space complexities of sequence alignment have been proposed. Hirschberg [7] first proposed a linear space algorithm for computing longest common subsequences. Myers and Miller [11] applied the Hirschberg's technique to Gotoh's algorithm [6]. After applying the Hirschberg's technique, the space requirement of pair-wise sequence alignment is reduced from $O(n^2)$ to $O(n)$ and introduce a small constant (about 2) slowdown to the $O(n^2)$ run time. Huang [9] extended this algorithm to local sequence alignment. Since the size of biological sequences could be very large, these algorithms are very important that make the pair-wise sequence alignment applicable. Although space-optimal algorithms make large sequence alignment practical, $O(n^2)$ run time still makes it time-consuming. Some parallel algorithms to reduce the computational cost have been proposed. Huang [8] presented a parallel pair-wise sequence alignment algorithm that uses optimal $O((m+n)/p)$ space complexity and suboptimal $O((m+n)^2/p)$ time complexity. Aluru et al. [1] presented another parallel algorithm with optimal $O((mn)/p)$ time complexity but uses $O(m+n/p)$ space complexity, where m, n are the lengths of input sequences and p is the number of processors. Afterward the optimal space and the time complexities are achieved, $O((m+n)/p)$ space complexity and $O((mn)/p)$ time complexity, parallel pair-wise sequence alignment algorithm was presented by Stjepan Rajko and Srinivas Aluru [14].

Huang [10] extended the Myers and Miller's algorithm to global three-sequence alignment with affine gap penalties that simultaneously aligns three sequence by using dynamic programming approach to find an optimal alignment in $O(n^2)$ space complexity and $O(n^3)$ time complexity. In practice, both $O(n^2)$ space complexity and $O(n^3)$ time complexity limit its applicability. In this paper, we apply a parallel method to Huang's algorithm [10] to reduce the space and time complexities.

3 Method

In this section, we first formalize the three-sequence alignment problem. After that we describe a dynamic programming (DP) algorithm with divide-and-conquer technology for solving three-sequence alignment problem which proposed by Huang [10]. We present the parallel three-sequence alignment algorithm in the end.

3.1 Three-sequence alignment problem

Let $A = a_1, a_2, \ldots, a_m$, $B = b_1, b_2, \ldots, b_n$, and $C = c_1, c_2, \ldots, c_l$ be three sequences over an alphabet Σ. Let '-', a unique symbol not in Σ, denote a gap.

Definition 1: An aligned triple consists of three ordered elements in $\Sigma \cup \{-\}$.

Definition 2: An alignment of A, B and C is a finite sequence of aligned triples. For each triple, the first element is always from sequence A or a gap ('-'), the second element is always from B or a gap and the third element is always from C or a gap.

Definition 3: A null triple is consisted of three gaps.

We only consider alignments without null triple. There are 7 types non-null aligned triples according to the number and the position of appearances of gaps ('-'). An example of three-sequence alignment is illustrated in Figure 1.

Definition 4: A block in an alignment is a contiguous subsequence of aligned triples of the same type bounded by aligned triples of other types or the end of the alignment.

Definition 5: An i-gap block is a block consisting of triples in which gap appears exactly i times.

There are only 1-gap blocks and 2-gap blocks in an alignment of three sequences. Given a scoring function $f : \Sigma \times \Sigma \rightarrow R$ that assign a value to each combination of two elements in alphabet Σ of a triple and a gap penalty function, the score of an alignment is the sum of the values of each aligned triple assigned by f minus gap penalties. (We can simply define f as $f(a, a) = 10$ for each a in Σ, $f(a, b) = -20$ for all a, b in Σ with $a \neq b$. For protein sequences, the f is usually defined through a scoring matrix such as PAM250.)

For gap penalties, an affine gap penalty function is commonly used. Let q_1 be a gap-open penalty for each 1-gap block and r_1 is a gap-extension penalty for each triple of 1-gap block, where q_1 and r_1 are two non-negative numbers. The affine gap penalty function takes p_1 for a triple of 1-gap block, where p_1 is defined as:

$$p_1 = \begin{cases} q_1 + r_1, \text{ if this triple is the start of the block.} \\ r_1, \text{ otherwise.} \end{cases}$$

We also define q_2, r_2 and p_2 for 2-gap block in the same way. Let x be the score of a triple. For any a, b and c in Σ, we have

$$x(a,b,c) = f(a,b) + f(a,c) + f(b,c)$$

$$x(a,b,-) = x(a,-,b) = x(-,a,b) = f(a,b) - p_1$$

$$x(a,-,-) = x(-,a,-) = x(-,-,a) = -p_2$$

```
A = GHBVADTHFASDFDAE
B = GSBVADTFASDAEE
C = GHBDTTHFASDDAEE

Alignment of A, B and C:

G-HBVADT-HFASDFDAE-
GS-BVADT--FASD--AEE
G-HB--DTTHFASD-DAEE
```

Figure 1: An alignment of three sequences. Each column in the alignment is an aligned triple.

$$S(i,j,k)=\begin{cases}0 & ,\text{if } i=0, j=0 \text{ and } k=0\\ -(q_2+r_2*i) & ,\text{if } i>0,\ j=0 \text{ and } k=0\\ -(q_2+r_2*j) & ,\text{if } i=0,\ j>0 \text{ and } k=0\\ -(q_2+r_2*k) & ,\text{if } i=0,\ j=0 \text{ and } k>0\\ \max\{G(i,j,k),H(i,j,k),I(i,j,k)\} & ,\text{if } i>0,\ j>0 \text{ and } k=0\\ \max\{E(i,j,k),H(i,j,k),J(i,j,k)\} & ,\text{if } i>0,\ j=0 \text{ and } k>0\\ \max\{F(i,j,k),I(i,j,k),J(i,j,k)\} & ,\text{if } i=0,\ j>0 \text{ and } k>0\\ \max\begin{cases}E(i,j,k),F(i,j,k),G(i,j,k),\\ H(i,j,k),I(i,j,k),J(i,j,k),\\ S(i-1,j-1,k-1)+x(a_i,b_j,c_k)\end{cases} & ,\text{if } i>0,\ j>0 \text{ and } k>0\end{cases}$$

$$E(i,j,k)=\begin{cases}S(i,j,k)-q_1 & ,\text{if } i=0 \text{ or } k=0\\ \max\{E(i-1,j,k-1),S(i-1,j,k-1)-q_1\}+f(a_i,c_k)-r_1 & ,\text{if } i>0 \text{ and } k>0\end{cases}$$

$$F(i,j,k)=\begin{cases}S(i,j,k)-q_1 & ,\text{if } j=0 \text{ or } k=0\\ \max\{F(i,j-1,k-1),S(i,j-1,k-1)-q_1\}+f(b_j,c_k)-r_1 & ,\text{if } j>0 \text{ and } k>0\end{cases}$$

$$G(i,j,k)=\begin{cases}S(i,j,k)-q_1 & ,\text{if } i=0 \text{ or } j=0\\ \max\{G(i-1,j-1,k),S(i-1,j-1,k)-q_1\}+f(a_i,b_j)-r_1 & ,\text{if } i>0 \text{ and } j>0\end{cases}$$

$$H(i,j,k)=\begin{cases}S(i,j,k)-q2 & ,\text{if } i=0\\ \max\{H(i-1,j,k),S(i-1,j,k)-q_2\}-r_2 & ,\text{if } i>0\end{cases}$$

$$I(i,j,k)=\begin{cases}S(i,j,k)-q2 & ,\text{if } j=0\\ \max\{I(i,j-1,k),S(i,j-1,k)-q_2\}-r_2 & ,\text{if } j>0\end{cases}$$

$$J(i,j,k)=\begin{cases}S(i,j,k)-q2 & ,\text{if } k=0\\ \max\{J(i,j,k-1),S(i,j,k-1)-q_2\}-r_2 & ,\text{if } k>0\end{cases}$$

Figure 2: Recurrences of the dynamic programming.

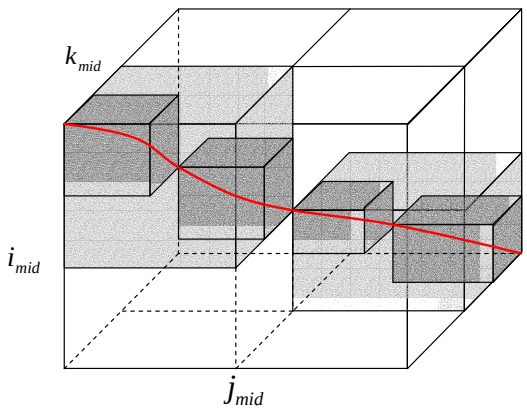

Figure 3: Schematic diagram of divide-and-conquer approach: After middle point $(i_{mid}, j_{mid}, k_{mid})$ is determined, the problem is divided into two subproblems (light gray areas), each of which will be further divided into two smaller subproblems (dark gray areas).

The score of an alignment is the sum of the score x of each triple. For example, the score of the alignment in Figure 1 is 246 if $f(a, a) = 10$ for each a in Σ, $f(a, b) = -20$ for all a, b in Σ with $a \neq b$, $q_1 = q_2 = 12$ and $r_1 = r_2 = 2$. The goal of three-sequence alignment problem is to find an alignment with best score, which is called an optimal alignment.

3.2 Space-efficient three-sequence alignment algorithm

Once a scoring mechanism is given, the optimal alignment can be found by using dynamic programming approach. Let $S(i, j, k)$ be the score of an optimal alignment of $A_{1,i}$, $B_{1,j}$ and $C_{1,k}$. The $S(i, j, k)$ can be computed along with auxiliary matrices according to the recurrences shown in Figure 2., where the matrices E, F and G save the scores that a gap opens at position j of B, i of A, and k of C, respectively. The matrices H, I and J save the scores that position i of A, j of B and k of C match two-gaps, respectively. Once $S(m, n, l)$ is computed, an optimal alignment of best score $S(m, n, l)$ can be found by a trace back procedure. Since the entire matrices have to be kept, the trace back procedure requires O(mnl) space. Note that it only takes four two dimensional matrices and a few one dimensional arrays if only the best score $S(m, n, l)$ is needed. For reducing the space requirement to O(mn), the divide-and-conquer technique of Hirschberg is applied. The central idea is to determine a middle point $(i_{mid}, j_{mid}, k_{mid})$ on an optimal path from $S(0, 0, 0)$ to $S(m, n, l)$ so that the original three-sequence alignment problem can be divided into two smaller three-sequence alignment problems; then these smaller three-sequence alignment problems are divided recursively. Finally, the optimal three-sequence alignment is obtained by merging the series of the computed middle points. Figure 3 illustrates this idea. For details, recall that $S(i, j, k)$ is the score of an optimal alignment of $A_{1,i}$, $B_{1,j}$ and $C_{1,k}$, the matrices introduced above are computed in an increasing order of indices. Another set of matrices can be defined symmetrically. Let R, U, V, W, X, Y, and Z be the set of matrices which are computed in a decreasing order of indices, where R corresponds to S, U to E, V to F and so on. Therefore, $R(i, j, k)$ is the score of an optimal alignment of $A_{i+1,m}$, $B_{j+1,n}$ and $C_{k+1,1}$ and $R(0,0,0)$, equal to $S(m, n, l)$, is the score of an optimal alignment of A, B and C. To find the middle point, first set k_{mid} to $\left\lfloor \frac{l}{2} \right\rfloor$. In the forward pass, compute the matrices S through J for $0 \le i \le m$, $0 \le j \le n$ and $0 \le k \le k_{mid}$, and save four two dimensional matrices $S(i, j, k_{mid})$, $E(i, j, k_{mid})$, $F(i, j, k_{mid})$ and $J(i, j, k_{mid})$ for $0 \le i \le m$ and $0 \le j \le n$. Note that k_{mid} is a character in sequence C and we do not have to consider the cases in which k_{mid} is a gap. In the reverse pass,

compute the matrices R through Z for $0 \le i \le m$, $0 \le j \le n$ and $0 \le k \le k_{mid}$, and save four two dimensional matrices $R(i, j, k_{mid})$, $U(i, j, k_{mid})$, $V(i, j, k_{mid})$ and $Z(i, j, k_{mid})$ for $0 \le i \le m$ and $0 \le j \le n$. The middle point of an optimal alignment of A, B and C is one which has the score

$$\max\begin{Bmatrix} S(i, j, k_{mid}) + R(i, j, k_{mid}), \\ E(i, j, k_{mid}) + U(i, j, k_{mid}) + q_1, \\ F(i, j, k_{mid}) + V(i, j, k_{mid}) + q_1, \\ J(i, j, k_{mid}) + Z(i, j, k_{mid}) + q_2 \end{Bmatrix} \text{ for } 0 \le i \le m \text{ and } 0 \le j \le n$$

Note that we need to add a gap-open penalty q_1 or q_2 to some pairs of matrix because an extra q_1 or q_2 is charged when two gaps of the same type merged into a single gap. Once the middle point $(i_{mid}, j_{mid}, k_{mid})$ is found, we recursively compute an optimal alignment of $A_{1,}\ i_{mid}$, $B_{1,}\ j_{mid}$ and $C_{1,}\ k_{mid}$ and that of $A_{i_{mid}+1,m}$, $B_{j_{mid}+1,n}$ and $C_{k_{mid}+1,l}$, respectively. The optimal three-sequence alignment is obtained by merging the series of the computed middle points.

3.3 Parallel algorithm

The critical cost of three-sequence alignment algorithm is the part of computation of the two dimensional matrices. Hence, we parallel this part to reduce computational time and space requirement. For each two dimensional matrices in three-sequence alignment, we divide these matrices into p parts, where p is the number of processors. Our parallel algorithm can be divided into four parts: initiation and allocation, forward pass, reverse pass and determine a middle point. The algorithm of initiation and allocation part is shown below:

Initiation and allocation:

Step 0: *Initialize p processors from rank_*0 *to rank_p*-1.

Step 1: *Read input sequences A, B and C, and compute the lengths M, N and L, respectively.*

Step 2: *set* $k_{mid} = \left\lfloor \frac{L}{2} \right\rfloor$.

Step 3: *Allocate 8 (M+1)*$\times(\left\lfloor \frac{N}{p} \right\rfloor$*+1) matrices and a few arrays for processor from rank_*0 *to rank_p*-2.

End ***Initiation and allocation***

In this part, we set k_{mid} to $\left\lfloor \frac{L}{2} \right\rfloor$ and dynamically allocate memory to each processor. Since input sequences may be large, dynamically allocate memory is a very important part. Recall that to implement divide-and-conquer technique, the matrices computation is divided into two pass, forward pass and reverse pass. The following two parts are used to parallel compute the eight two dimensional matrices, *S, E, F, J, R, U, V* and *Z*. These matrices store the values computed according to the recurrences defined before. The forward pass, which handles matrices *S, E, F and J,* is shown below:

Forward pass:

For k=0 *to* $k=k_{mid}$ **do**
Processor ***rank_*0**:
  Step 0: *Compute all two dimensional matrices.*
  Step 1: *Send the last columns of each two dimensional matrices to rank_*1.
Processor ***rank_(i),*** **for** $1 \le i \le p-2$**:**
  Step 0: *Receive the last columns of each matrices from rank_(i*-1*).*
  Step 1: *Compute all two dimensional matrices.*
  Step 2: *Send the last columns of each two dimensional matrices to rank_(i*+1*).*
Processor ***rank_p*-1:**
  Step 0: *Receive the last columns of each matrices from rank_p*-2.
  Step 1: *Compute all two dimensional matrices.*
End for

End ***Forward pass***

The reverse pass, which handles matrices *R, U,* V and Z, is shown below:

***Reverse pass*:**

For $k=L$ *to* $k=k_{mid}$ **do**
Processor ***rank_p-1*:**
  Step 0: *Compute all two dimensional matrices.*
  Step 1: *Send the last columns of each two dimensional matrices to rank_p*-2.
Processor ***rank_(i),*** **for** $1 \le i \le p-2$**:**
  Step 0: *Receive the last columns of each matrices from rank_(i*-1*).*
  Step 1: *Compute all two dimensional matrices.*
  Step 2: *Send the last columns of each two dimensional matrices to rank_(i*+1*).*
Processor ***rank*_0:**
  Step 0: *Receive the last columns of each matrices from rank_p*-2.
  Step 1: *Compute all two dimensional matrices.*
End for

End ***Reverse pass***

After above two passes are done, we merge these eight matrices to find a middle point. For each processor, find a middle point and send it to processor *rank_*0 as a candidate middle point. Processor *rank_*0 determine a middle point with the

best score with these candidate middle points. This part is shown below:

Determine a middle point **(i_{mid}, j_{mid}, k_{mid})**

For processor *Rank*_1 to *Rank_p*-1
Step 0: *Find a middle point.*
Step 1: *Send middle point to Rank_0.*
For processor *Rank_0:*
Step 0: *Find a middle point as a candidate point.*
Step 1: *Receive middle points from other processors as candidate middle points.*
Step 2: *Determine the middle point among these candidate middle points*

End *Determine a middle point* **(i_{mid}, j_{mid}, k_{mid})**

4. Analysis

In this section, the time and space complexities of our parallel algorithm will be proved. The advantages and disadvantages are also analyzed.

In each $(m+1)\times(n+1)$ matrices, each processor handles exactly $m*(\left\lfloor \frac{n}{p} \right\rfloor +1)$ elements except last (rank_*p-1*) processor. The last processor handles at most $2\times m\times(\left\lfloor \frac{n}{p} \right\rfloor +1)$ elements. Obviously each processor takes $O((mn)/p)$ time complexity and $O((mn)/p)$ space complexity in each $(m+1)\times(n+1)$ matrices. Since space-efficient algorithm is applied, for input sequences of length *m*, *n* and *l*, each processor takes $O((mnl)/p)$ time complexity and $O((mn)/p)$ space complexity.

In the following, we analyze the performance of the *p l*-stages pipelines. We can see that the last processor have to wait for previous *p*-1 processors to finish their job in first matrix. It takes about one matrix computation (including communication) time. After that the pipeline is full loaded. In the last matrix the first processor have to wait about one matrix computation (including communication) time to finish whole computation. For three sequences with the length of 1000 base pairs, the full loaded ratio of the pipeline is (1000/(1000+2))*100% = 99.8%, hence, most of the time the pipeline is full loaded.

5. Experimental Results

Our algorithm has been implemented by *MPI+C*, and tested on the *NCHC Formosa Linux Cluster*. A random data set with complete match and mismatch cases, as used in [14], is used to evaluate our algorithm. The runtime of our parallel program with various numbers of processors and lengths of input sequences is summarized in Table 1. From Table 1, we can see that a good performance and a good speed-up are achieved by our parallel algorithm, which is more applicable than sequential version. The speed-up is similar to that achieved by [14] for the parallel pair-wise sequence alignment method.

6. Conclusions

In this paper, we have proposed a parallel algorithm for three-sequence alignment problem. In order to evaluate our parallel algorithm, a random data set has been used. Our experimental results demonstrate that our parallel algorithm is more practical and applicable than the sequential version. Moreover, our parallel algorithm can achieve a good speed up.

References

[1]. S. Aluru, N. Futamura, and K. Mehrotra, "Parallel biological sequence comparison using prefix computations," *J. Parallel and Distributed Computing*, , vol. 63, pp. 264-272, 2003.

[2]. P. Bonizzoni and G. D. Vedova, "The complexity of multiple sequence alignment with SP-score that is a metric," *Theoretical Computer Science*, vol. 259, pp. 63-79, 2001.

[3]. H. Carrillo and D. Lipman, The multiple sequence alignment problem in biology, *SIAM Journal on Applied Mathematics*, vol. 48, pp. 1073-1082, 1988.

[4]. S.C. Chan, A.K.C. Wong and D.K.Y. Chiu, "A survey of multiple sequence comparison methods," *Bulletin of Mathematical Biology*, vol. 54, pp. 563-598, 1992.

[5]. D.Gusfield, *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology.* Cambridege University Press, NY., 1997.

[6]. O. Gotoh, "An improved algorithm for matching biological sequences," *J. Molecular Biology*, vol. 162, pp. 705-708, 1982.

[7]. D.S. Hirschberg, A linear space algorithm for computing maximal common subsequences, *Communications of the ACM*, vol. 18, pp. 341-343, 1975.

[8]. X. Huang, "A Space-Efficient Parallel Sequence Comparison Algorithm for a Message-Passing Multiprocessors," *Int'l j. Parallel Programming*, vol. 18, no. 3, pp. 223-239, 1989.

[9]. X. Huang, "A Space-Efficient Algorithm for Local Similarities," Computer Applications in the Biosciences, vol. 6, pp.373-381, 1990.

[10]. X. Huang, "Alignment of Three Sequences in Quadratic Space," *ACM SIGAPP Applied Computing*, vol. 1, pp. 7-11, 1993.

[11]. E.W. Myers and W. Miller, "Optimal Alignments in Linear Space," *Computer Applications in the Biosciences*, vol. 4, pp. 11-17, 1988.

[12]. H.B. Nicholas, A.J. Ropelewski and D.W. Deerfield, "Strategies for multiple sequence alignment," *Biotechniques*, vol. 32, pp. 592-603, 2002.

[13]. C. Notredame, Recent progresses in multiple sequence alignment: a survey. *Pharmacogenomics*, vol. 3, pp. 131-144, 2002.

[14]. S. Rajko and S. Aluru, "Space and Time Optimal Parallel Sequence Alignments," *IEEE Transactions on Parallel and Distributed Systems*, vol. 15, pp. 1070-1081, 2004.

[15]. J. Setubal and J. Meidanis, *Introduction to Computational Molecular Biology.* PWS Publishing Company, 1997.

[16]. L. Wang and T. Jiang, "On the complexity of multiple sequence alignment," *Journal of Computational Biology*, vol. 1, pp. 337–348, 1994.

Appendix

Table 1: The execution time for various numbers of processors and lengths of input sequences.

No. of processors / **sequences size**	1	2	4	8	16	32
Complete match case						
1k×1k ×1k	85	64	37	21	15	11
2k×2k ×2k	737	495	263	145	88	75
3k×3k ×3k	2384	1702	925	468	256	283
4k×4k ×4k	5745	3965	2079	1076	582	522
5k×5k ×5k	11096	7748	4063	2098	1154	643
Complete mismatch case						
1k×1k ×1k	126	90	47	26	21	23
2k×2k ×2k	1022	714	366	193	105	107
3k×3k ×3k	3441	2411	1228	635	342	294
4k×4k ×4k	8161	5673	2906	1482	777	635
5k×5k ×5k	16285	11314	6227	2923	1519	865

Time: second.